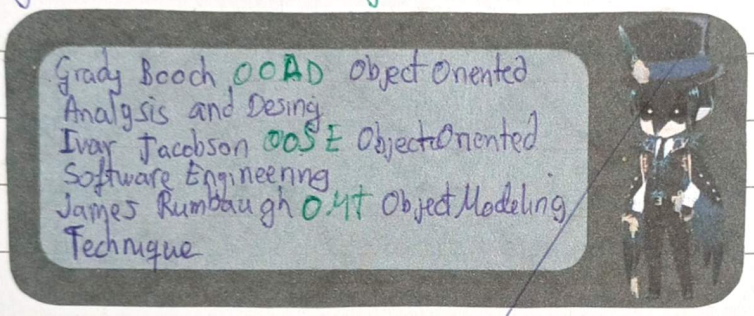
Capítulo 1

Introducción a la Programación orientada a Objetos

Antecedentes

* Programación Convencional (caótica) al estar en desorden, no se entendía
* Programación Estructurada – E. Dijkstra – 70
  Todo problema se resuelve con 3 estructuras de control: Secuencial, Condicional, Repetitiva
  Prohibido el uso de la sentencia GO TO
* Programación Modular (Funciones y procedimientos)
  Reutilizar código
* Programación Orientada a Objetos (90)
  Los 3 amigos Booch, Rumbaugh, Jacobson



* Todo es objeto real o abstracto
  Ejemplo lapiz, persona, vaso, computadora, telefono, galleta, lentes, televisor oso de peluche taxi casa (real)
  numeros operadores clima figuras geometricas software personajes de videojuegos (abstracto)

Se deben identificar características de los objetos

Lápiz: color, tipo, marca, tamaño, color de envoltura)
Persona: nombre, apellidos, género, peso, estatura, color de piel, color de ojos, tipo de cabello, nacionalidad, CI, edad
Vaso: tipo, color, material, RAM, color, procesador, modelo

Nota: Depende el contexto se añaden o descartan características

MOOVING LOOP

Escaneado con CamScanner

Personaje de Videojuegos nombre, genero, raza, nivel, equipamiento

Los objetos se agrupan en conjuntos llamados clases

Conjunto de personas

Clase "Persona"

Caracteristicas generales
nombre, apellido, edad, genero, color de piel

obj 1 ← "pepe" "murillo" 20 "varon" "trigueño"
obj 2 ← "ana" "gomez" 18 "mujer" "blanca"
obj 3 ← "luis" "ba" 9 "varon" "trigueño"

Dentro de la clase persona, se instancian los objetos obj 1, obj 2, obj 3

Representación de algoritmos

- Algoritmo Conjunto de pasos para resolver un problema
- Programa Conjunto de instrucciones (dadas en un lenguaje de Programación) para resolver un problema

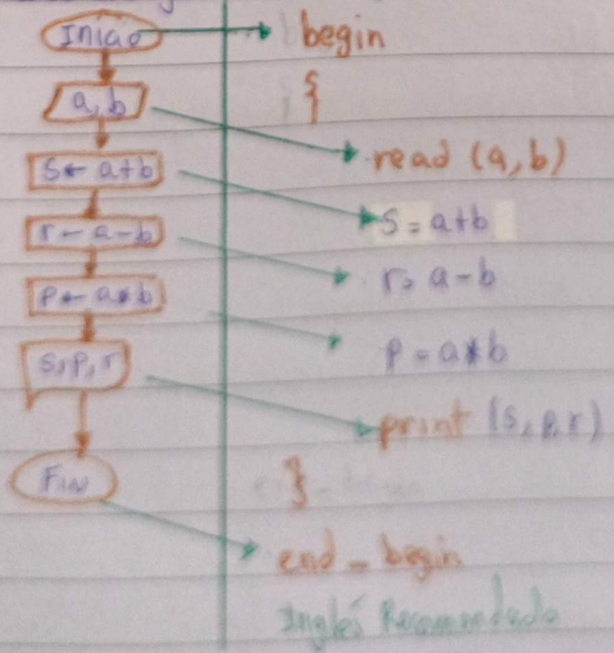
Para representar algoritmos:

Lenguaje natural

Diagramas N-S

Diagrama de Flujo

Pseudo código (Pasi código)




```
begin
{
    read (a,b)
    S = a+b
    r = a-b
    P = a*b
    print (S, P, r)
}
end - begin
```

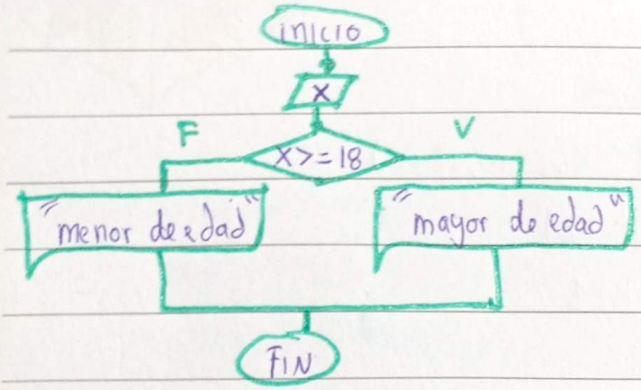
Inglés Recomendado

```
Inicio
{
    leer (a, b)
    S = a+b
    r = a-b
    P = a*b
    mostrar (a,b)
}
fin - inicio
```

Español (No recomendado

MOOVING LOOP

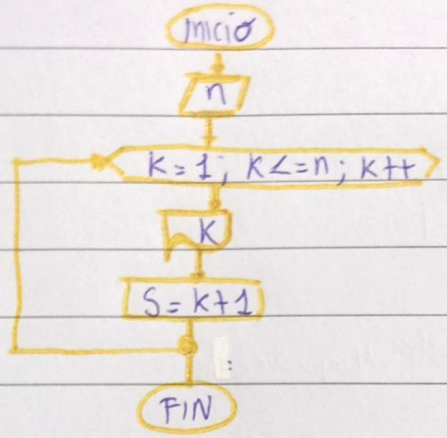
Escaneado con CamScanner



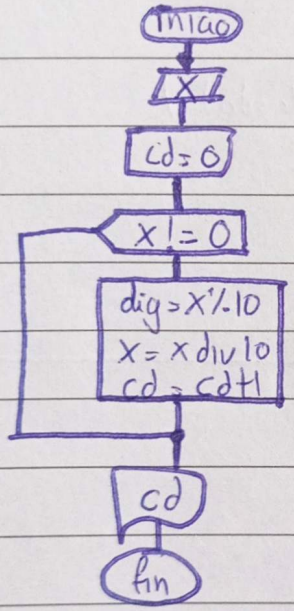

```
begin
{
    read (x)
    if (x>=18)
    {
        print("Mayor de edad")
    }
    else
    {
        print ("Menor de edad")
    }
    end_if
} end_begin
```

```
begin {
    read (x)
    if (x>=18)
        print ("Mayor de edad")
    else
        print ("Menor de edad")
    end_if
} end_begin
```

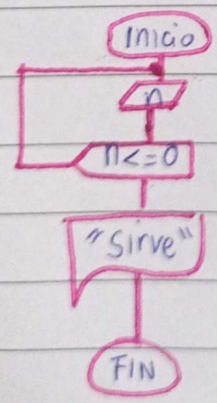
Escaneado con CamScanner




```
begin {
    read (n)
    for (k=1, K<=n, K++)
        print (k)
        S = k+1
    end_for
} end_begin
```

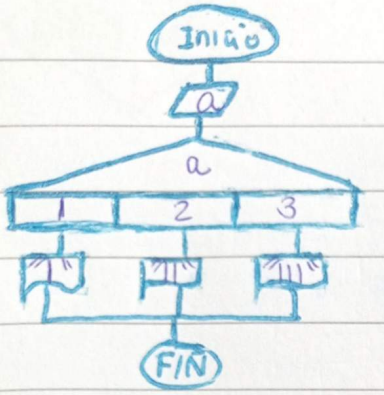



```
begin {
    read (x)
    cd=0
    while (x!=0)
        dig = X%10
        X = X div 10
        cd = cd+1
    end_while
    print (cd)
} end_begin
```

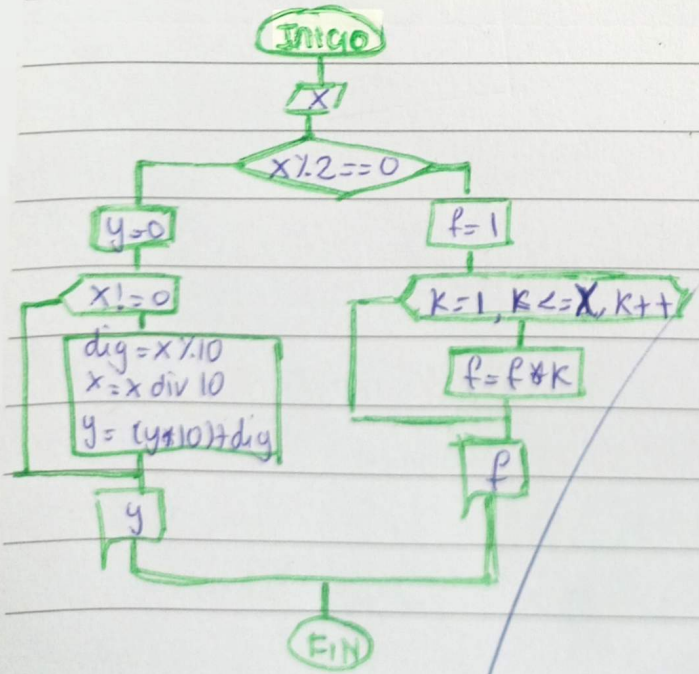



```
begin {
    do
        read (n)
    while (n<=0)
    print ("Sirve")
} end_begin
```

Escaneado con CamScanner



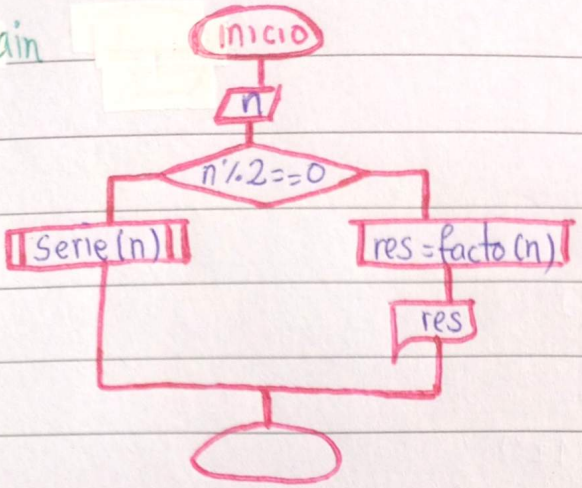

```
Begin {
    read (a)
    Switch (a)
        case 1:
            print ("I")
            break
        case 2:
            print ("II")
            break
        case 3:
            print ("III")
            break
        default:
            print "Fuera de Rango
    end_switch
} end_begin
```

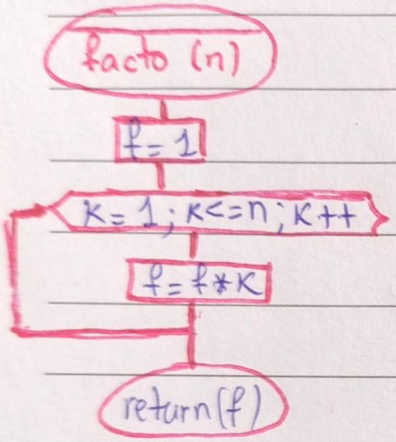



```
Begin {
    read (x)
    if (x%2==0)
        f=1
        for (k=1; k<=x; k++)
            f= f*k
        end_for
        print (f)
    else
        y=0
        while (x!=0)
            dig=x%10
            x=x div 10
            y=(y*10)+dig
        end_while
        print(y)
    end_if
```

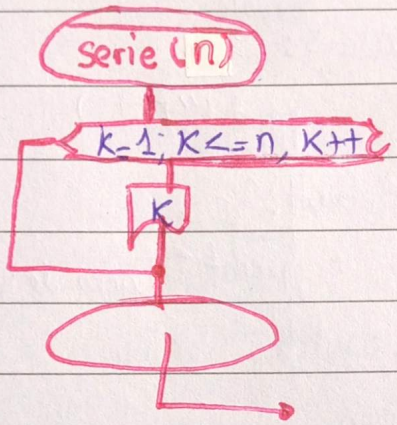
Escaneado con CamScanner

Main





Función



Procedimiento

```
Begin_Main {
    read (n)
    if (n%2==0)
        res = facto(n)
        print (res)
    else
        serie (n)
    end-if
} end-begin-main
```

Escaneado con CamScanner

```
facto (n)
    f=1
    for(k=1; k<=n; k++)
        f=f*k
    end_for
    return(f)
end. facto
```

```
serie (n)
    for (k=1, k<=n, k++)
        print (k)
    end_for
end_serie
```

```
Begin
    a: int
    b: int      } declaración de variables
    c: int
    read (a,b)
    s=a+b
    print (s)
End_begin
```

tipos de datos

numérico

int	entero	3, -20	
long	entero largo	123366888758	
float	flotante	5,6	-0,7845
double	real largo	0,12345678899888555554	

texto

char	caracter	'S'	'_' '2'
C# string / java String	cadena	"hola 3 # hola"	

verdad falso boolean booleano (V/F)

Escaneado con CamScanner

Pilares de la POO

abstracción Sacar las características para modelar una clase

encapsular definir que tipo de acceso tendrá una clase y quienes pueden usar

herencia se heredan características de una clase padre a las clase hijos

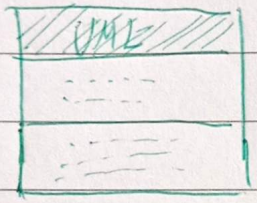
polimorfismo se adoptan varias formas de una clase

La abstracción depende del observador

Diagramas de las Programación Orientada a Objetos

Los 3 amigos

Lenguaje de Modelado Unificado (UML)



Entender bien el uml y respetar el modelo

Ej abstracción una anciana ve un gato con pelaje y una veterinaria ve esqueleto de un gato

Ej encapsulación dormitorio privado, entra el dueño, sala es público

Ej herencia un hijo hereda la nariz del padre

Ej polimorfismo una persona habla, un perro ladra, un gato maulla pero hablan todos

Clase Conjunto de objetos con características similares

Objeto es una instancia/elemento de una clase (es un elemento particular de la clase

Notación Para representar clases vamos a usar el diagrama de clases

Nombre de la clase

Atributos de la clase

(características comunes)

Metodos de la clase

Operaciones / procedimientos y Funciones

Escaneado con CamScanner

Reglas

- El nombre de la clase, inicia en mayúscula y se escribe en singular

  Persona, Mesa, Estudiante, País

Si se necesitan 2 o más para representar una clase, se utilizan la notación Camel Case

Para Figuras Geometricas → FiguraGeometrica

Para Ser humano → SerHumano

Atributos de la clase Las caracteristicas comunes que tiene los objetos que pertenecen a la clase

Inician en Mayuscula

Para 2 o más palabras usar camelCase

Ejemplo nombre, edad, estadoCivil, fechaNacimiento

Se debe identificar el tipo de dato

int, float, double, long, char, String, etc

Metodos de la clase Son las operaciones que se pueden hacer con los objetos de la clase (con los atributos) se implementan a través de funciones y procedimientos

inician en minusculas

Si se necesitan 2 o más palabras, camel case

reciben entradas o no entre parentesis

actualizarEdad(), cambiarNombre(nuevo), leer()

El objeto invoca o llama a los métodos después de un punto

sc.nextFloat() System.out.print() Console.ReadLine()

Ejemplo

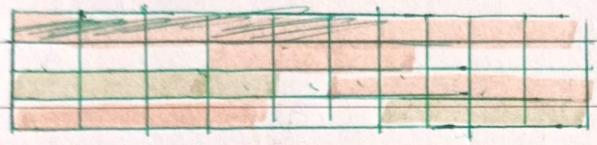
Persona
- nombre: String - apellidoPaterno: String - apellidoMaterno: String - edad: int - nacionalidad: String - genero: String
+ Persona() + leer(): void + mostrar(): void + cambiarNombre(nuevo: String): void + actualizarEdad(e: int): int

MOOVING

Escaneado con CamScanner

Constructor El que ayuda a crear o instanciar objetos en la memoria del computador. Puede ser con o sin argumentos

Destructor Permite liberar memoria ocupada por un objeto en la memoria



espacio ocupado
espacio libre
espacio hecho por el destructor

El destructor (garbage collector) ya viene en los lenguajes

El constructor lleva el mismo nombre de la clase y no tiene tipo de dato Persona()

Constructor: Persona(), Persona(h,l,l)

Destructor: ~Persona()

Encapsulamiento

Modificadores

- Público (public) cuando un atributo o metodo puede ser usado, modificado accedido dentro o fuera de la clase ("+")
- Privado (Private): cuando un atributo o metodo puede ser usado SOLO al interior de la clase ("-")
- Protegido (protected): Para herencia ("#")

Ejemplo

Persona
- nombre: String - apellidoPaterno: String - apellidoMaterno: String - edad: int - nacionalidad: String - genero: String
+ Persona() + leer: void + mostrar: void + cambiarNombre(nuevo: String): void + actualizarEdad(e: int): int

Escaneado con CamScanner

```
class Persona
    private:
        nombre : String
        apellidoPaterno : String
        apellidoMaterno : String
        edad : int
        nacionalidad : String
        genero : String
    public:
        Persona ()
        this.nombre = ""
        this.apellidoPaterno = ""
        this.apellidoMaterno = ""
        this.edad = 0
        this.nacionalidad = ""
        this.genero = ""
        end_Persona
        leer () : void
            read (nombre)
            read (apellidoPaterno)
            read (apellidoMaterno)
            read (edad)
            read (nacionalidad)
            read (genero
        end_leer ()
```

Escaneado con CamScanner

Solo se usa el this para cuando dos atributos se llaman igual

```
    mostrar ( ): void
        print (nombre)
        print (apellido Paterno)
        print (apellido Materno)
        print (edad)
        print (nacionalidad)
        print (genero)
    end_ mostrar ()
    cambiar Nombre (nuevo: String): void
    this. nombre = nuevo
    end_ cambiar Nombre
    actualizar Edad (edad: int): int
        this.edad = this.edad + edad
        return (this.edad)
    end_ actualizar Edad
} end_ Persona
Begin }
    // int a;
    // a = 3
    // print (a)
    Persona x = new Persona ()
    x.leer ()
    x.mostrar ()
    Persona y;
    y = new Persona ()
    y.leer ()
    y.mostrar ()
} end - begin
```

Escaneado con CamScanner

Fracción
- numerador : int - denominador : int + Fraccion ( ) + leer ( ) : void + mostrar ( ) : void + sumarFraccion (a: Fraccion, b: Fraccion) : Fraccion + SumarFracion 2 (x : Fraccion) : Fraccion

```
class Fracción
{
    private
        numerador : int
        denominador : int
    public :
        Fracción ( )
            numerador = 0
            denominador = 1
        end-Fracción ( )
        leer ( ) : void
            read (this. numerador)
            read (this. denominador)
            do
                read (this. denominador)
            while (this. denominador == 0)
        end_leer ( )
        mostrar : void
            print (numerador + "/" + denominador)
        end_mostrar ( )
        Sumar Fracción (a: Fracción, b: Fracción) : Fracción
            Suma : Fracción
            suma = new Fracción
```

MOOVING LOOP

Escaneado con CamScanner

Suma.numerador = (a.numerador * b.denominador)
+ (a.denominador * b.numerador)

Suma.denominador = (a.denominador * b.denominador)

return (Suma)

end_sumar Fracción

sumar Fracción 2 (x: Fracción): Fracción

suma : Fracción

Suma: new Fracción

Suma.numerador = (this.numerador * x.denominador) +
(this.denominador * x numerador)

Suma.denominador = (this.denominador * x.denominador)

return (Suma)

end_sumar Fracción 2

end_Fracción

Para usar 1 solo dato en particular, se utilizan los getters y los setters. ubicadas en la parte pública de la clase

// getters (obtienen 1 solo atributo
+ getNombre(): String
+ getApellidoPaterno(): String
+ getApellidoMaterno(): String
+ getEdad(): int
+ getNacionalidad(): String
+ getGenero(): String
// setters (Ponen 1 valor en 1 atributo)
+ setNombre(nom: String): void
+ setApellidoPaterno(pat: String): void
+ setApellidoMaterno(mat: String): void
+ setEdad(e: int): void
+ setNacionalidad(nac: String): void
+ setGenero(gen: String): void

Escaneado con CamScanner

```
// getters
getNombre (): String
    return (this.nombre)
end_getNombre
getApellidoPaterno (): String
    return (this.apellidoPaterno)
end_getApellidoPaterno
getApellidoMaterno (): String
    return (this.apellidoMaterno)
end_getApellidoMaterno
getEdad (): int
    return (this.edad)
end_getEdad
getNacionalidad (): String
    return (this.nacionalidad
end_getNacionalidad
getGenero (): String
    return (this.genero)
end_getGenero
// setters
setNombre (nom: String): void
    this.nombre = nom
end_setNombre
setApellidoPaterno (pat: String): void
    this.apellidoPaterno = pat
end_setApellidoPaterno
setApellidoMaterno (mat: String): void
    this.apellidoMaterno = mat
end_setApellidoMaterno
```

MOOVING LOOP

Escaneado con CamScanner

```
set Edad (e: int): void
    this.edad = e
end_set Edad
set Nacionalidad (nac: String) void
    this.nacionalidad = nac
end_set Nacionalidad
set Genero
    this.genero = gen
end_ set Genero
```

Ejercicio

Colegio
- nombre: String - fundacion: int - direccion: String - tipo: String - turno: String
+ Colegio() + leer(): void + mostrar(): void + getNombre(): String + getFundacion(): int + getDireccion(): String + getTipo(): String + getTurno(): String + setNombre(nom: String) + setFundacion(fun: int) + setDireccion(dir: String) + setTipo(tip: String) + setTurno(tur: String) + calcularAnios(): int + compararColegios(col2: Colegio): boolean + unirColegios(col1: Colegio, col2: Colegio): Colegio

Escaneado con CamScanner

```
Class Colegio {
  private:
    nombre : String
    fundación: int
    direcáon String
    tipo : String
    turno : String
  public:
    Colegio()
      this.nombre = ""
      this.fundación = 0
      this.direcáon = ""
      this.tipo = ""
      this.turno = ""
    end_Colegio
    leer(): void
      read (this.nombre)
      read (this.fundación)
      read (this.direcáon)
      read (this.tipo)
      read (this.turno)
    end_leer
    mostrar(): void
      print (this.nombre)
      print (this.fundacion)
      Print (this.direcáon)
      Print (this.tipo)
      Print (this.turno
    end_mostrar
```

MOOVING LOOP

Escaneado con CamScanner

```
getNombre(): String
    return (this.nombre)
end_getNombre
getFundación(): int
    return (this.fundación)
end_getFundación
getDirección(): String
    return (this.dirección)
end_getDirección
getTipo(): String
    return (this.tipo)
end_getTipo
getTurno(): String
    return (this.turno)
end_getTurno
setNombre(Nom: String): void
    this.nombre = nom
end_setNombre
setFundación(Fun: int): void
    this.fundación = fun
end_setFundación
setDirección(Dir: String): void
    this.dirección = dir
end_setDirección
setTipo(Tip: String): void
    this.tipo = tip
end_setTipo
```

© & ™ WBEI (s24)

MOOVING LOOP

Escaneado con CamScanner

```
setTurno (tur: String): void
    this.turno = turn
end_setTurno
calcularAnios (): int
    res : int
    res = 2026 - this.fundacion
    return (res)
end_calcularAnios
compararColegios (col2: Colegio): boolean
    if (this.nombre == col2.nombre and this.fundacion == col2.fundacion
        and this.direccion == col2.direccion and this.tipo == col2.tipo and
        this.turno == col2.turno)
        return true
    else
        return false
    end_if
end_compararColegios
unirColegios (col1: Colegio, col2: Colegio): Colegio
    colegioNuevo: Colegio
    colegioNuevo = new Colegio()
    colegioNuevo.nombre = col1.nombre + col2.nombre
    colegioNuevo.fundacion = 2026
    colegioNuevo.direccion = col1.direccion + col2.direccion
    colegioNuevo.tipo = "convenio"
    colegioNuevo.turno = col1.turno + col2.turno
    return (colegioNuevo)
end_unirColegios
} end_Colegio
```

Escaneado con CamScanner

```
begin {
    Colegio a = new Colegio()
    Colegio b = new Colegio()
    a.leer()
    b.leer()
    print(a.calcularAnios())
    print(b.calcularAnios())
    if (a.compararColegios(b))
        print("iguales")
    else
        print("distintos")
    end_if
} end_begin
```

PersonajeVideoJuego
- nombre: String - vida: int - habilidad: String
+ PersonajeVideoJuego() + leer(): void + mostrar(): void getNombre(): String getVida(): int getHabilidad(): String setNombre(nom: String): void setVida(v: int): void setHabilidad(hab: String): void aumentarVida(inc: int): void aumentarHabilidades(ch: String): void compararPersonajes(per2: PersonajeVideoJuego): Boolean

Escaneado con CamScanner

```
class PersonajeVideoJuego ( ) {
    private:
        nombre String
        vida int
        habilidad String
    public:
        PersonajeVideoJuego ()
            this.nombre = ""
            this.vida = 0
            this.habilidad = ""
        end_PersonajeVideoJuego
        leer ( ): void
            read (this.nombre)
            read (this.vida)
            read (this.habilidad)
        end_leer ( )
        mostrar ( ): void
            print (this.nombre)
            print (this.vida)
            print (this.habilidad)
        end_mostrar ( )
        getNombre ( ): String
            return (this.nombre)
        end_getNombre
        getVida ( ): int
            return (this.vida)
        end_getVida
```

MOOVING LOOP

Escaneado con CamScanner

getHabilidad(): String
    return (this.habilidad)
end_getHabilidad
setNombre(nom: String): void
    this.nombre = nom
end_setNombre
setVida(v: int): void
    this.vida = v
end_setVida
setHabilidad(hab: String): void
    this.habilidad = hab
end_setHabilidad
aumentarVida(inc: int): void
    this.vida = this.vida + inc
end_aumentarVida
aumentarHabilidad(h: string): void
    this.habilidad = this.habilidad + h
end_aumentarHabilidad
compararPersonajes(per2: PersonajeVideoJuego): boolean
    if (this.nombre == per2: PersonajeVidooJuego and this.vida == per2.vida
        and this.habilidad == per2.habilidad)
        return (true)
    else
        return (false)
    end_if
end_compararPersonajes

© & ™ WBEI (s24)

MOOVING LOOP

Escaneado con CamScanner

Polimorfismo Los métodos pueden tomar distintas formas, pero se usa la misma palabra
pagar impuesto de auto, de casa, caracteristicas distintas, pero son impuestos
hablar

Muchas formas:

Los metodos pueden tomar muchas formas llamandose igual

Los diferencia su comportamiento interno, entre ellos

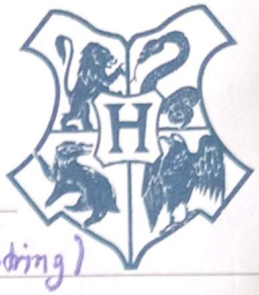
Cantidad y Tipo de argumentos ()

En polimorfismo existen 2 formas de trabajar

- Sobrecarga de metodos (Java, C#)
  - leer ()
  - leer (n: int)
  - Suma (2, 3)
  - Suma (4, 7, 8, 4, 7)
- Sobrecarga de operadores (C#)
  - +, −, *, /, %, ==, >, <
  - + para sumar numeros
  - + para concatenar o unir cadenas
  - A + B
  - Persona 1 + persona 2
  - Frac 1 + Frac 2

MOOVING LOOP

Escaneado con CamScanner



+ Persona ( ) // Constructor SIN argumentos

+ Persona (n: String, p: String, m: String, e: int, nac: String, g: String)

// Constructor sobrecargado CON argumentos

Persona ( )

this. nombre = ""

this. apellido Paterno = ""

this. apellido Materno = ""

this. edad = 0

this. nacionalidad = ""

this. genero = ""

end_persona( )

Persona (n: String, p: String, m: String, e: int, nac: String, g: String)

this. nombre = n

this. apellido Paterno = p

this. apellido Materno = m

this. edad = e

this. nacionalidad = nac

this. genero = g

end_persona( )

Colegio ( )

this. nombre = ""

this. fundación = 0

this. direcación = ""

this. tipo = ""

this. turno = ""

end_Colegio ( )

MOOVING LOOP

© & ™ WBEI (s24)

Escaneado con CamScanner

Colegio (nom: String, fun: int, dir: String, tip: String, tur: String)

this. nombre = nom

this. fundacion = fun

this. dirección = dir

this. tipo = tip

this. turno = tur

end - Colegio

Vivienda
- folio - dirección - superficie
+ Vivienda () + Vivienda (fo: String dir: String, sup: String) + Vivienda (fo: String) + leer ( ): void + mostrar ( ): void + get Folio ( ): String + get Direcion: String + get Superficie: int + set Folio (f: String): void + set Direcón (d: String): void + set Superficie (s: int): void + Calcular Impuesto ( ): double + calcular Impuesto (factor: double): Double + comparar Folio (f: String): boolean + comparar Viviendas (v2: Vivienda): boolean + comparar Viviendas (fo: String): boolean + comparar Viviendas (v1: vivienda, v2: vivienda

```
Class Vivienda {

    private:
        folio: String
        dirección: String
        superficie: int

    public:
        Vivienda ( )
            this. folio = ""
            this. folio = ""
            this. superficie = 0
        end - Vivienda ( )
```

MOOVING LOOP

Escaneado con CamScanner

```
Vivienda (fo: String, dir String, sup int)
    this.folio = fo
    this.direccion = dir
    this.superficie = sup
end_Vivienda
Vivienda (fo: String)
    this.folio = fo
    this.direccion = ""
    this.superficie = 0
end_Vivienda
leer (): void
    read (this.folio)
    read (this.direcion)
    read (this.superficie)
end_leer
mostrar (): void
    print (this.folio)
    print (this.direccion)
    print (this.superficie)
end_mostrar
getFolio (): String
    return (this.folio)
end_getFolio
getDireccion: String
    return (this.direccion)
end_getDireccion
getSuperficie
    return (this.superficie)
end_getSuperficie
```

MOOVING
LOOP

Escaneado con CamScanner

```
setFolio (f: String): void
    this.folio = f
end_setFolio
setDireccion (d: String): void
    this.dirección = d
end_setDirecaón
setSuperficie (s: int): void
    this.superficie = s
end_SetSuperficie
calcularImpuesto (): double
    imp: double
    imp = this.superficie * (1,5)
    return (imp)
end_calcularImpuesto
calcularImpuesto (factor: double): double
    imp: double
    imp = this.superficie * factor
    return (imp)
end_calcularImpuesto
comprobarFolio (f: string): boolean
    if (this.folio = f)
        return (true)
    else
        return (false)
    end_if
end_comprobarFolio
```

MOOVING LOOP

Escaneado con CamScanner

```
compararViviendas(v2:Vivienda): boolean
    if (this.folio == v2.folio) and (this.dirección == v2.dirección)
        and (this.superficie == v2.superficie)
        return true
    else
    end_if
end_compararViviendas
compararViviendas(fo:String): bodean
    if (this.folio == fo)
        return (true)
    else
        return (false)
    end_if
end_compararViviendas
compararViviendas(v1:Vivienda, v2:Vivienda): boolean
    if (v1.folio == v2.folio)
        return (true)
    else
        return (false)
    end_if
end_compararViviendas
```

3 end_vivienda

Tipos de operadores

- Unarios requieren 1 solo valor para operar K++, !x
- Binarios requieren 2 valores para operar

Suma : a+b

Suma : 2+5

A>B

100 > 150

MOOVING LOOP

Escaneado con CamScanner

Vector
-n: int -vec[]: int
+ Vector() + Vector(n: int) + leer(): void + mostrar(): void + getN(): int + getVec(): int[] + setN(n: int): void + setV(v[]: int): void + operator +(v1: Vector, v2: Vector): Vector + operator *(a: Vector, b: Vector): Vector

```
class Vector {
    private:
        n: int
        vec[]: int
    public:
        Vector()
            this.n=0
            dim this.vec[n]
        end_Vector
        Vector(n:int)
            this.n=n
            for (k=0; k<this.n; k++)
                this.vec[k]=0
            end_for
        end_Vector
        leer(): void
            read(this.n)
            for (k=0, k<this.n, k++)
                read (this.vec[k])
            end_for
        end_leer
```

MOOVING LOOP

Escaneado con CamScanner

```
mostrar():void
    print(this.n)
    for(K=0; K<this.n, K++)
        print(this.vec[K])
    end_for
end_mostrar
getN():int
    return(this.n)
end_getN
//getVec NO aplica
setN(n:int):void
    this.n=n
end_setN
setVec(v[]:int):void
    this.vec=v  //ojo 1a1
end_setVec
operator + (V1: Vector, v2:Vector): Vector
    Vector res= new Vector()
    res.n = v1.getN()
    for(K=0; K<v1.getN; K++)
        res.vec[K] = V1.vec[K]+V2.vec[K]
    end_for
    return(res)
end_operator +
// aquiva el operator *
}end_vector
```

MOOVING LOOP

Escaneado con CamScanner

Rectangulo
- base : double - altura : double - area : double - penmetro : double
+ Rectangulo () + Rectangulo (b: double, a: double) + leer () : void + mostrar () : void + getBase () : double + getAltura () : double + getArea () : double + getPenmetro () : double + SetBase (b: double) : void + setAltura (a: double) : void + setArea (area: double) : void + SetPenmetro (p: double) : void + verificar Mayor (rec2 : Rectangulo) : boolean + verificar Mayor (rec2 : Rectangulo, rec3 : Rectangulo) : boolean + operator + (rec1 : Rectangulo, rec2 : Rectangulo) : Rectangulo + operator * (rec1 : Rectangulo, rec2 : Rectangulo) : Rectangulo + operator == (rec1 : Rectangulo, rec2 : Rectangulo) : boolean + operator ! = (r1 : Rectangulo, r2 : Rectangulo) : boolean

```
class Rectangulo
  private
      base : double
      altura : double
      area : double
      penmetro : double
  public
      Rectangulo ()
          this.base = 0
          this.altura = 0
          this.area = 0
          this.penmetro = 0
      end_Rectangulo ()
```

MOOVING LOOP

Escaneado con CamScanner

```
Rectangulo ( b:double, a:double)
    this.base=b
    this.altura=a
    this.area= this.base * this.altura
    this.penmetro= 2*(this.base)+(2* this.altura)
end_Rectangulo()
leer():void
    read (this.base)
    read (this.altura)
    this.area = this.base * this.altura
    this.perimetro= 2*this.base + 2 * this.altura
end_leer()
mostrar(): void
    print (this.base)
    print (this.altura)
    print (this.area)
    print (this.penmetro)
end_mostrar()
getBase():double
    return (this.base)
end_getBase
getAltura():double
    return (this.altura)
end_getAltura
getArea():double
    return (this.area)
end_getArea
```

© & ™ WBEI (s24)

MOOVING LOOP

Escaneado con CamScanner

```
getPerimetro():double
    return
end_getPerimetro
setBase(b:double):void
    this.base=b
end_setBase
setAltura(a:double):void
    this.altura=a
end_setAltura
setArea(area:double):void
    this.area=area
end_setArea
setPerimetro(p:double):void
    this.perimetro=p
end_setPerimetro
verificarMayor(rec2:Rectangulo):boolean
  if this.area > rec2.area
    return true
  else
    return false
  end_if
end_verificarMayor
verificarMayor(rec2:Rectangulo, rec3:Rectangulo):boolean
  if this.area > rec2.area and this.area > rec3.area)
    return true
  else
    return false
  end_if
end_verificarMayor
```

MOOVING LOOP

Escaneado con CamScanner

```
operator + (rec1:Rectangulo, rec2:Rectangulo):Rectangulo
    suma: Rectangulo
    suma = new Rectangulo ()
    suma.base = rec1.base + rec2.base
    suma.altura = rec1.altura + rec2.altura
    suma.area = suma.base * suma.altura
    suma.penmetro = (2*suma.base) + (2* suma.altura)
    return (suma)
end_operator +
opperator * (rec1: Rectangulo, rec2: Rectangulo): Rectangulo
    prod: Rectangulo
    prod = new Rectangulo
    prod.base = rec1.base * rec2.base
    prod.altura = rec1.altura * rec2.base
    prod.area = rec1.area * rec2.area
    prod.penmetro = rec1.penmetro * rec2.perimetro
    return (prod)
end_operator *
operator == (rec1:Rectangulo rec2:Rectangulo): boolean
    resultado: boolean
    if (rec1.base == rec2.base and rec1.altura = rec2.altura)
        resultado = true
    else
        resultado = false
    end_if
    return (resultado)
end_operator ==
```

©& ™ WBEI (s24)

MOOVING LOOP

Escaneado con CamScanner

```
operator != (r1: Rectangulo, r2: Rectangulo)
    resultado: boolean
    if (r1.base != r2.base and r1.altura != r2.altura)
        resultado = true
    else
        resultado = false
    end_if
    return (resultado)
end_operator =!
} end_Rectangulo
```

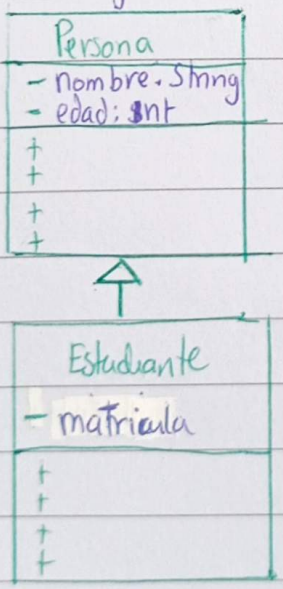
Operadores Relacionales == != >= <=

object.equals

Herencia

Permite aprovechar las características del padre para los hijos

- Clase Padre (clase Base) (clase)
- Clase Hijo (clase Subclase) (clase derivada)



MOOVING LOOP

Escaneado con CamScanner

Relaciones entre clases

- Herencia

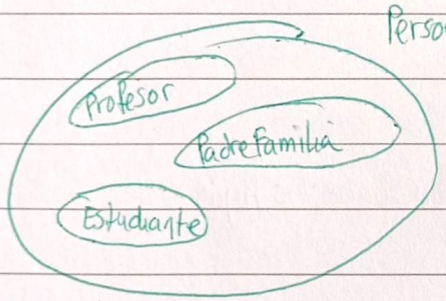
Herencia Simple (Java, C#

Reglas

- Siempre la flecha sale de la clase hijo hacia la clase padre
- Se heredan atributos y metodos
- La clase hijo asume que los atributos del padre le pertenecen (no se escriben) en el diagrama de clases

Ejemplo Colegio

- Profesores, estudiantes, padres de familia Son personas



© & ™ WBEI (s24)
MOOVING
Escaneado con CamScanner

```
class Persona {
    protected:
        ci: Stnng
        nombre: Stnng
    public:
        Persona()
            this.ci = ""
            this.nombre = ""
        end_Persona()
        Persona (c: stnng, nom: Stnng)
            this.ci = c
            this.nombre = n
        end_Persona()
        getCi(): Stnng
            return (this.ci)
        end_getCi()
        getNombre(): String
            return (this.nombre)
        end_getNombre
        setCi(c: Stnng): void
            (this.ci) = c
        end_setCi
        setNombre(nom: Stnng): void
            this.nombre. nom
        end_setNombre
```

MOOVING LOOP

Escaneado con CamScanner

```
    leer Persona()
        read this.ci
        read this.nombre
    end_leer Persona
    mostrar Persona()
        print (this.ci)
        print (this.nombre)
    end_mostrarPersona()
} end_Persona

class Profesor extends Persona {
    private
        especialidad: String
        item: String
    public
        Profesor()
            super() //en java: super, en c#: base
            this.especialidad = ""
            this.item = ""
        end_Profesor
        Profesor(ci:String nom String esp String item String
            super (ci, nom)
            this.especialidad = esp;
            this.item = item
        end_Profesor()
```

© & ™ WBEI (s24)

MOOVING

Escaneado con CamScanner

```
leerProfesor ( ) : void
    this.leerPersonal( ) // ojo esta en la clase padre
    read(this especualidad)
    read(this.item)
end_leerProfesor
mostrarProfesor ( ) : void
    this.mostrarPersona( ) // clase Padre
    print(this especualidad)
    print (this.item)
end_mostrarProfesor
getEspecualidad ( ): String
    return (this.especualidad)
end_getEspecualidad
getItem ( ): String
    return (this.item)
end_getItem
setEspecualidad (esp: String) : void
    this.especualidad = esp
end_setEspecualidad
setItem (item: String) : void
    this.item = item
end_setItem
calcularHonorarios: double
    sueldo: double
    if (this.especualidad == "matematicas"
        sueldo = 3500
    end if
```

MOOVING LOOP

Escaneado con CamScanner

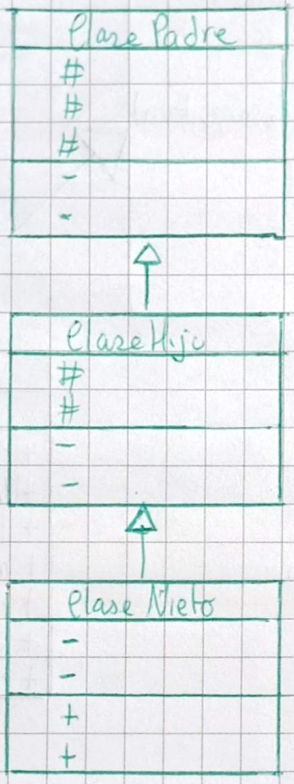
```
        if (this.especialidad == "sociales")
            sueldo = 3400
        end_if
        if (this.especialidad == "artes")
            sueldo = 3300
        end_if
        if (this.especialidad = "")
            print ("Error")
            sueldo = 0
        end_if
        return (sueldo)
    end_calcularHonorarios
} end_Profesor
```

Escaneado con CamScanner

Tema: Fecha: DÍA MES AÑO

Herencia

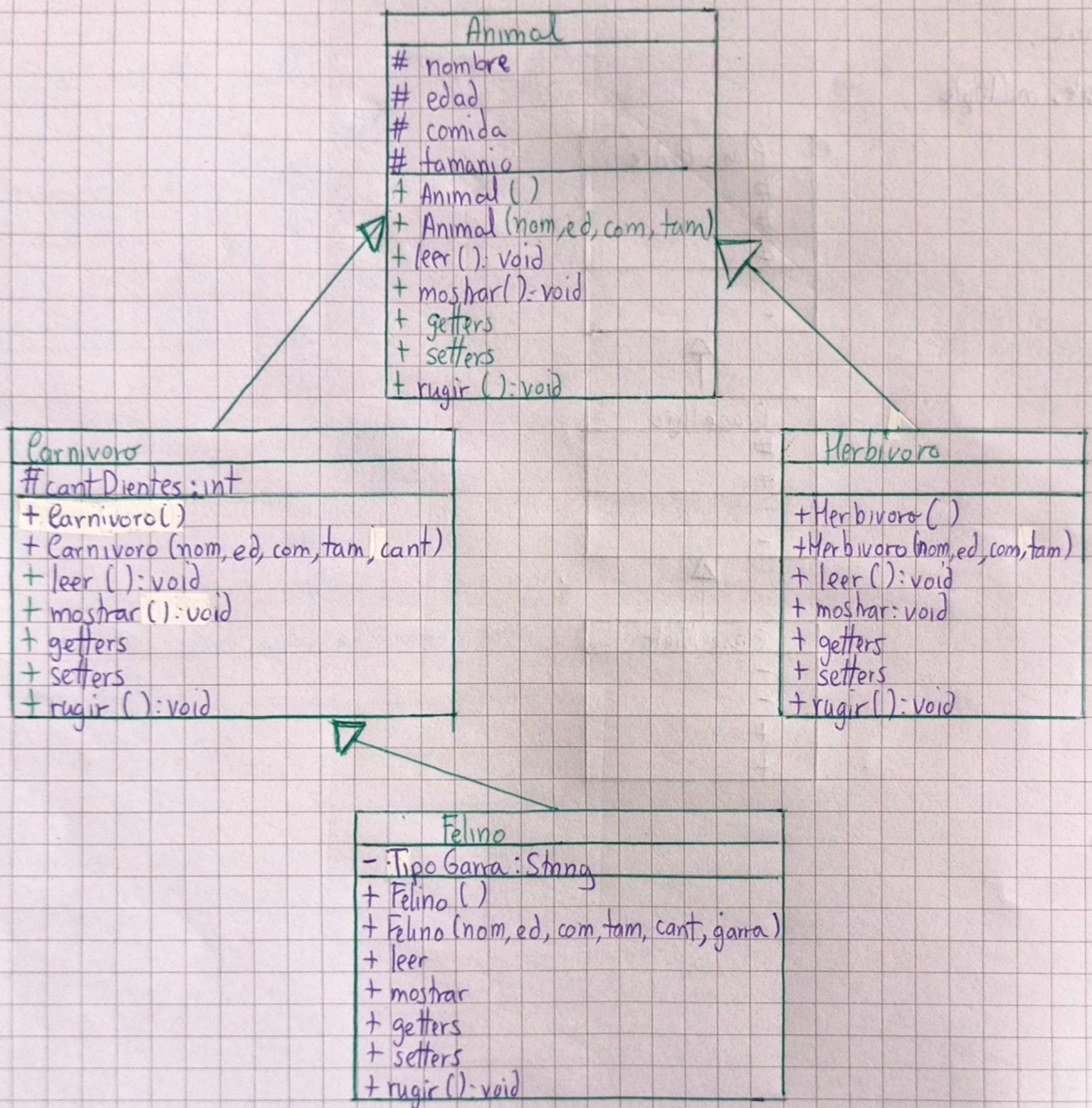
Simple, múltiple



PREMIER
Inmapel creando ideas

Escaneado con CamScanner

Tema: Fecha: DÍA MES AÑO

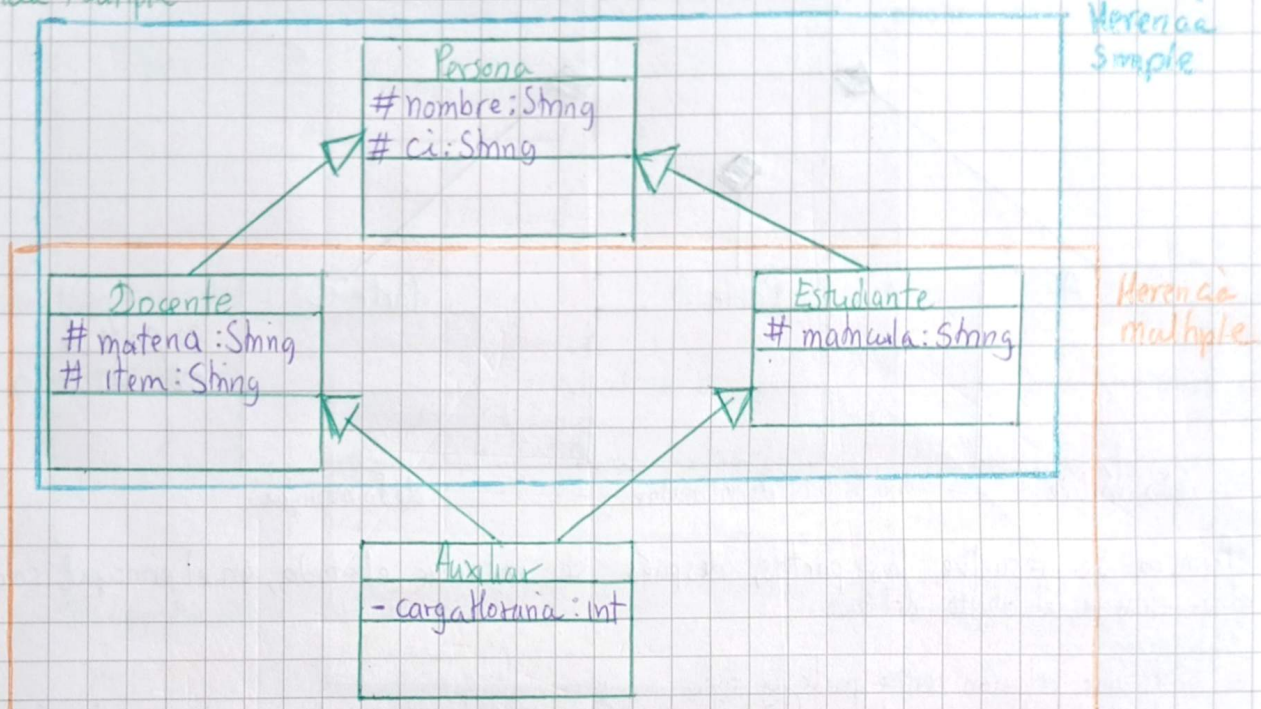


Inmapel creando ideas PREMIER

Escaneado con CamScanner

Tema: Fecha: DÍA MES AÑO

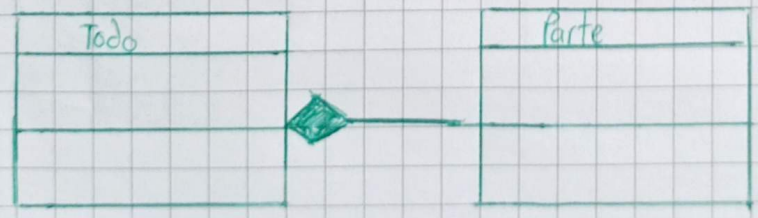
Herencia Multiple



Ya no soporta el Java ni el C#

Relaciones entre clases

- Herencia
  - Herencia Simple
  - Herencia Multiple
- Agregaciones y composiciones
  - Relaciones fuertes y debiles entre clases
    - Composicion (agregacion fuerte)
    - Del todo y sus partes

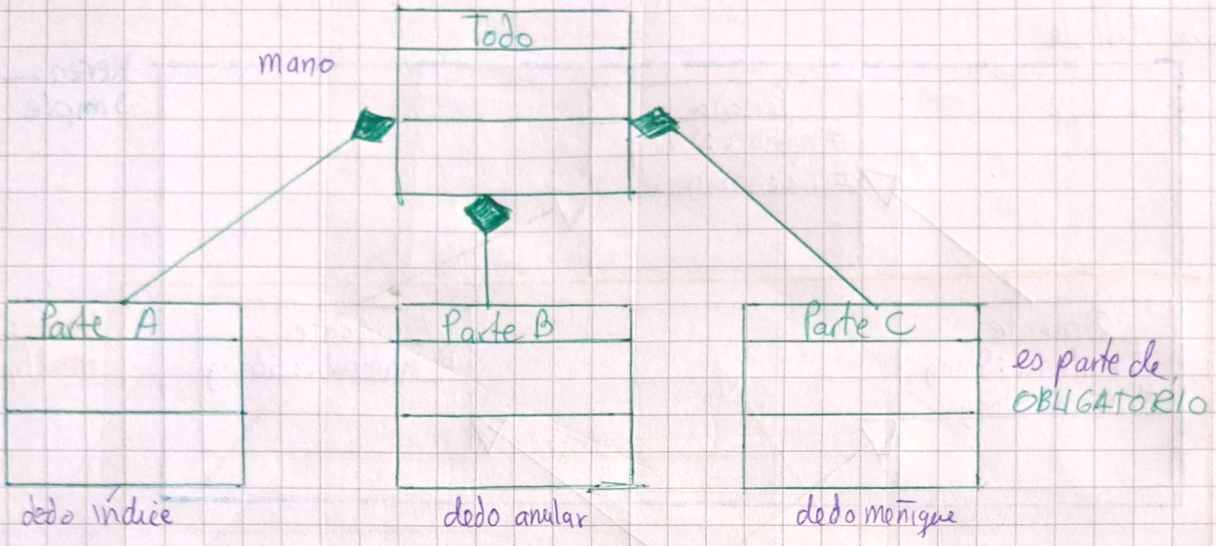


Las PARTES no existen sin el todo

Es obligatorio en la composición que el rombo esté pintado y que sale de la parte hacia el todo

Escaneado con CamScanner

Tema: Fecha: DÍA MES AÑO



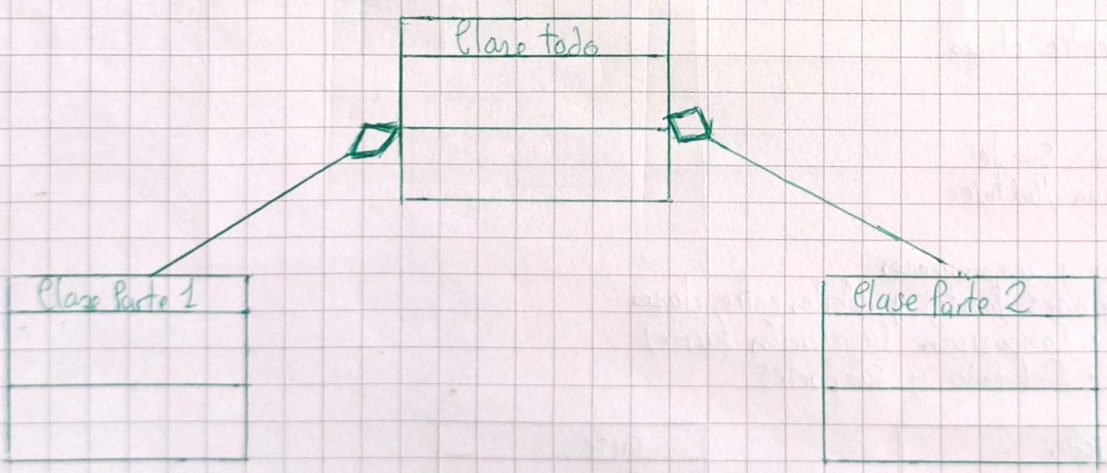
Primero se resuelve las partes, después se resuelve el todo, en el principal solo se debe generar el objeto del todo

Agregación

Es una relación entre parte y todo

No hay obligación de que las partes estén en el todo, las partes pueden existir de forma independiente al todo

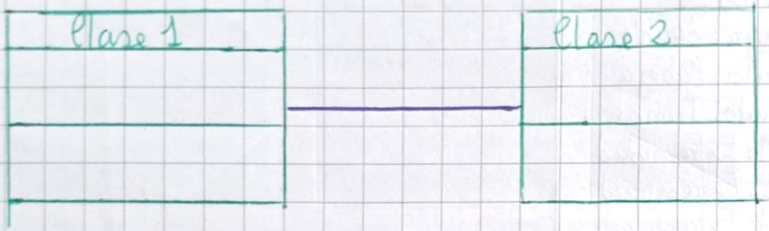
Usa flecha de diamante o rombo SIN pintar el rombo, debe ser blanco



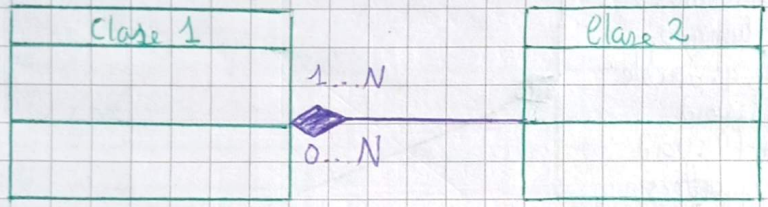
Escaneado con CamScanner

Tema: Fecha: DÍA MES AÑO

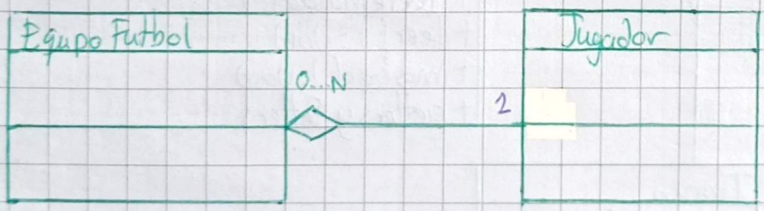
Asociación Es una relación más genérica entre clases



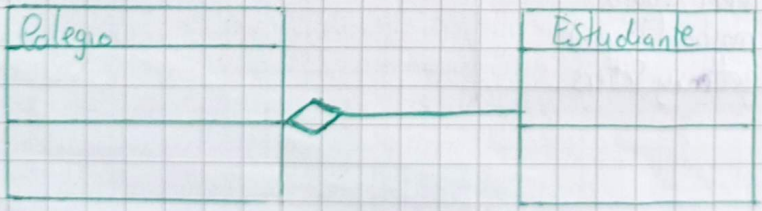
Cardinalidad o Multiplicidad



Cuerpo humano = Cabeza, relación a 1-1
Tronco, relación a 1-1
Extremidades, relación a 1-4

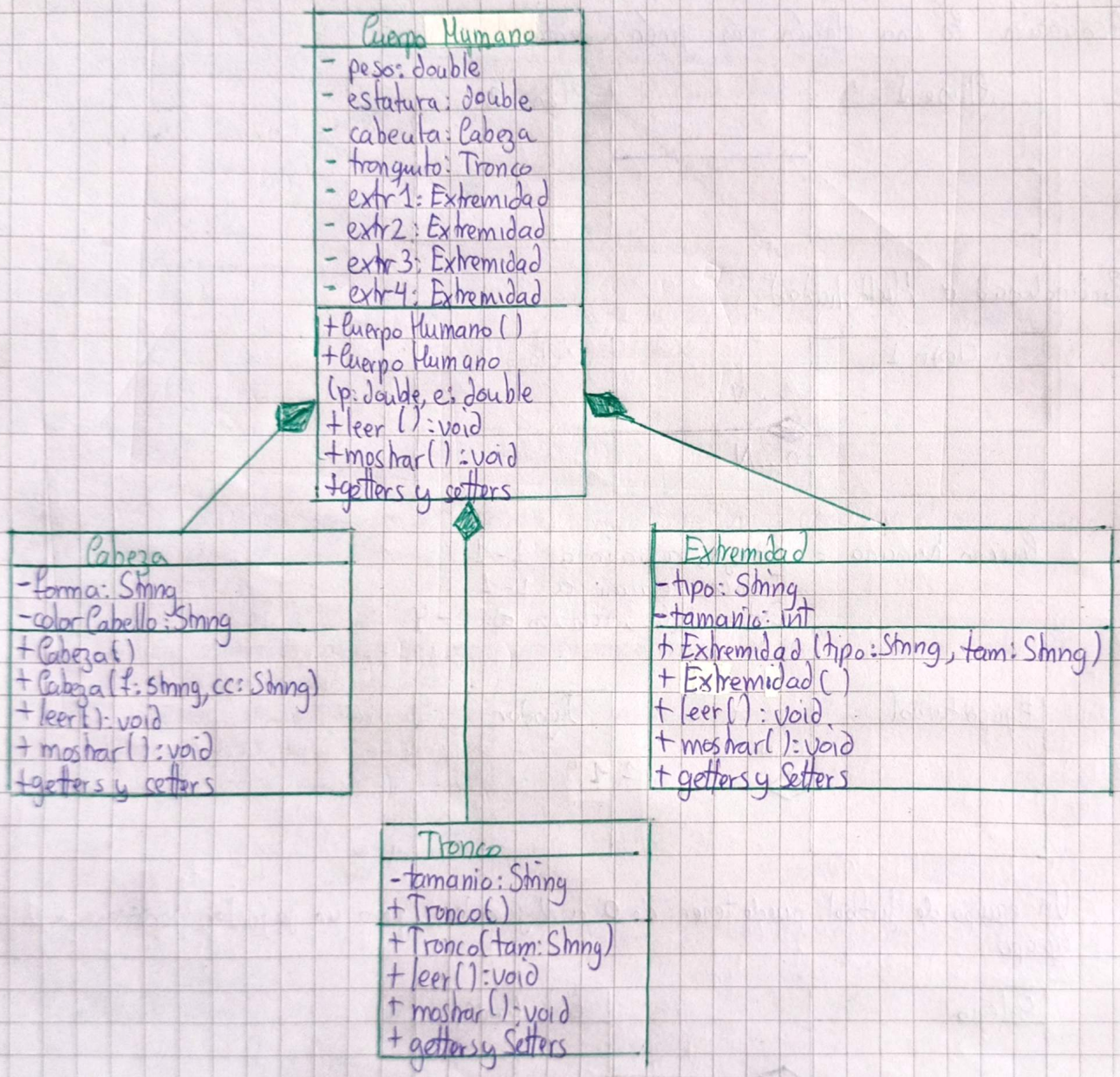


Un equipo de futbol puede tener de 0 a N jugadores, pero un jugador pertenece a 1 equipo



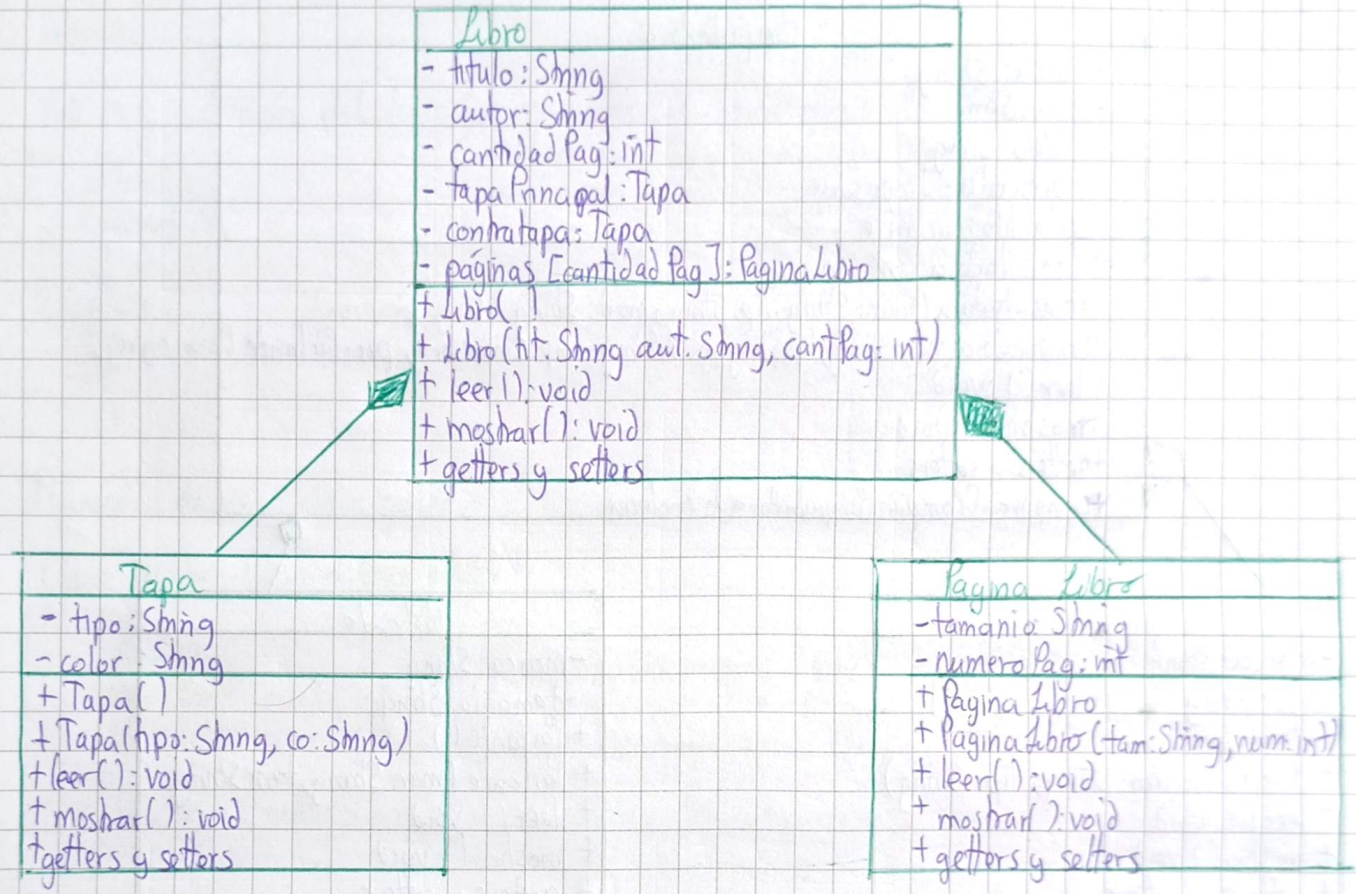
Escaneado con CamScanner

Tema: Fecha: DÍA MES AÑO



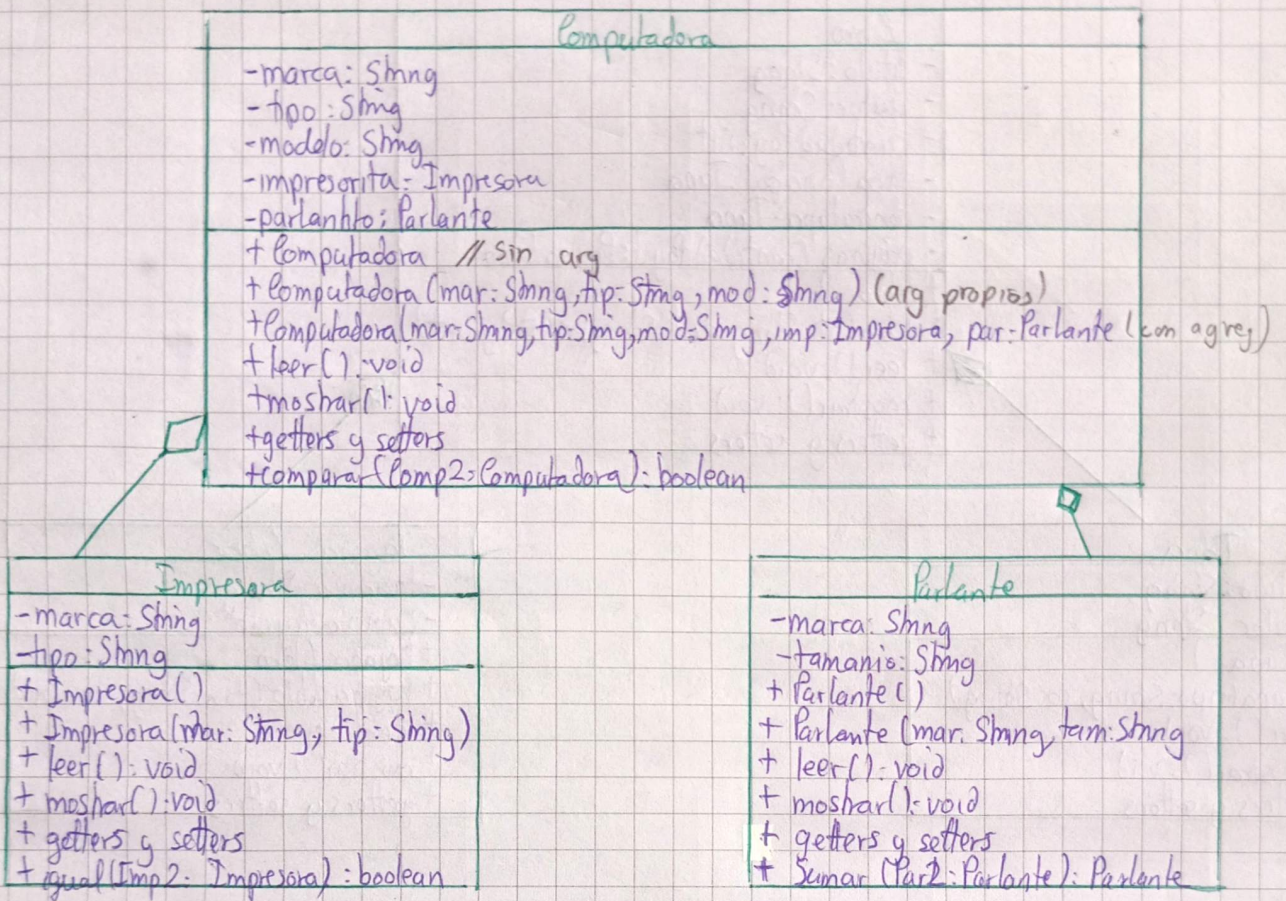
Inmapel "creando ideas"
PREMIER
Escaneado con CamScanner

Tema: Fecha: DÍA MES AÑO



PREMIER
Inmapel creando ideas
Escaneado con CamScanner

Tema: Fecha: DÍA MES AÑO



Inmapel creando ideas PREMIER

Escaneado con CamScanner

Tema: Fecha: DÍA MES AÑO

Genericidad

Es una extensión del lenguajes que permite parametrizar, los tipos de datos

- Clases
- Metodos

Beneficios

Definición de una clase genérica

```
Class Nombre Clase <Tipo dato>
{
    //Atributos
    Tipo Dato atributo 1;
    Tipo Dato atributo 2;
    //Metodos
    Tipo Dato metodo 1;
}
```

Creación del objeto de una clase genérica

```
Main {
    Nombre Clase <char> objeto = new Nombre Clase <char> ();
    Nombre Clase <int> obj 2 = new Nombre Clase <int> ();
}
```

PREMIER Inmapel creando ideas

Escaneado con CamScanner

Tema: Fecha:

Fracción: num / den

$\frac{2}{5}$ num1 a ✓ char
num2 b — char

Fracción <TD>
- num : TD
- den : TD
+ Fracción (num:TD, den: TD)
+ leer (a: TD, b: TD): void
+ mostrar (): void
+ getters y setters
+ igual (x: Fracción <TD>): boolean

Excepciones

Son anomalías producidas en tiempo de ejecución

Java usa excepciones para proveer capacidades de manejo de excepciones

Una excepción es un evento que ocurre en la comida de código que interrumpe el flujo de la ejecución

```
try {
  // codigo que puede generar excepciones
}
catch (ExceptionType1 e) {
  // acciones para manejar la excepción
} catch
  //
} finally
  // codigo que se ejecuta siempre, haya o no excepción
}
```

catch y finally son opcionales, pero mínimo tiene que haber un try

Utilidad

Separa el codigo normal de la excepción

Fraccion <int> a, new Fraccion <int> (3, 5)
<char> 'a', 'd'
<string> String ("uno", "dos")

Escaneado con CamScanner